Documentación técnica del scraper de eventos culturales de Chile

Arquitectura, operación y mantenimiento

Alcance:	extracción, normalización y sincronización del catálogo
Versión documental:	20 de octubre de 2026
Tecnologías principales:	Python, Playwright, NVIDIA NIM, SQLite y Supabase
Ejecución automatizada:	GitHub Actions

Índice

1. Propósito y alcance **2**

1.1. Principios de diseño 2

2. Arquitectura general **2**

2.1. Patrones aplicados 3

3. Estructura del repositorio **3**

4. Fuentes de datos **3**

4.1. Validación de candidatas 4

5. Flujo de procesamiento **5**

5.1. Fase de extracción 5

5.2. Fase de procesamiento y publicación 5

6. Modelo canónico de evento **5**

7. Saneamiento, vigencia y filtro cultural **6**

8. IA, OCR y control de cuota **6**

9. Estado local, checkpoints y recuperación **7**

10. Exportación y sincronización con Supabase **7**

10.1. Limpieza protegida y conciliación 7

11. Configuración **8**

12. Automatización con GitHub Actions **9**

12.1. Pruebas de base de datos 9

12.2. Cambios del 20 de octubre de 2026 10

13. Operación y mantenimiento **10**

13.1. Comandos frecuentes 10

13.2. Cuando una fuente cambia o cae 10

14. Seguridad, límites y consideraciones **11**

15. Documentos relacionados **11**

1. Propósito y alcance

El proyecto construye y mantiene un catálogo de eventos culturales de Chile a partir de fuentes públicas: agendas municipales, espacios culturales, universidades, portales sectoriales y tiqueteras. Su objetivo no es copiar una página completa, sino convertir información pública heterogénea en registros comparables, trazables y vigentes.

El scraper abarca cuatro responsabilidades:

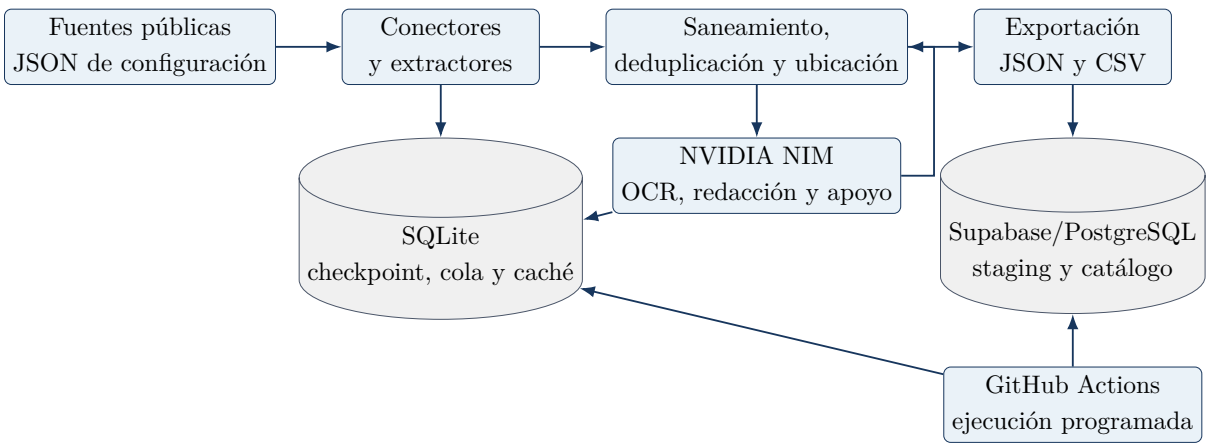
1. descubrir y extraer eventos desde fuentes configuradas;
2. normalizar fechas, ubicación, categorías, organizador y enlaces;
3. enriquecer solo los casos necesarios mediante OCR y redacción asistida por IA;
4. exportar un respaldo local y sincronizar el catálogo publicado con Supabase.

No forma parte de este documento el frontend, la autenticación de usuarios ni el algoritmo de recomendaciones. El scraper alimenta el catálogo que dichas capas consumirán posteriormente.

1.1. Principios de diseño

- **Trazabilidad:** cada evento conserva fuente, URL original, URL oficial cuando existe, método de extracción y momento de captura.
- **Preferencia por datos estructurados:** API, JSON-LD, WordPress REST y tarjetas declarativas se usan antes que OCR o modelos de IA.
- **Mínima presión sobre los sitios:** se respeta el ritmo por dominio, hay caché condicional HTTP y la concurrencia es acotada.
- **Idempotencia:** repetir una ejecución actualiza registros existentes y evita duplicados mediante identificadores estables y deduplicación.
- **Resiliencia:** una fuente caída, un afiche ilegible o una respuesta transitoria de NVIDIA no detienen el resto del proceso.
- **Vigencia:** se retiran eventos vencidos y registros ausentes de fuentes completas, respetando supresiones, interacciones y las protecciones de publicaciones manuales.

2. Arquitectura general



2.1. Patrones aplicados

Patrón	Aplicación
Strategy	Cada conector implementa una estrategia de extracción independiente para una familia de sitios.
Factory/Registry	`ConnectorFactory` selecciona el conector a partir del campo `connector` del JSON de fuente.
Repository	`SourceRepository` carga, valida y entrega la configuración declarativa de `sources/`.
Pipeline	`EventPipeline` orquesta fases en orden, sin acoplar la lógica a una página concreta.
Inyección de dependencias	HTTP, estado, exportador y servicios remotos pueden sustituirse en pruebas.

3. Estructura del repositorio

```
scrape_events.py Punto de entrada de la ejecucion
eventos/ Paquete de extraccion y procesamiento
  connectors/ Estrategias por tipo de fuente
  extractors/ Lectores reutilizables, por ejemplo JSON-LD
  services/ NVIDIA NIM y cliente REST de Supabase
  source_validation/ Pipeline independiente de validacion de fuentes
sources/ Un JSON por fuente y plantillas reutilizables
state/ SQLite local: checkpoint, cola de IA y cache
data/ Exportaciones JSON, CSV, metricas y auditorias
supabase/migrations/ Migraciones SQL versionadas
supabase/tests/ Regresiones SQL con datos ficticios
tools/ Utilidades, entre ellas copia consistente de estado
tests/ Pruebas unitarias y de integracion simulada
.github/workflows/ Automatizacion en GitHub Actions
docs/ Documentacion del scraper
```

Los directorios `state/`, `data/`, cachés y el archivo `.env` son locales y no se publican en Git. Las fuentes, las migraciones, las pruebas y el workflow sí deben versionarse.

4. Fuentes de datos

La configuración vigente contiene 21 fuentes permanentes y una fuente estacional, activa únicamente en septiembre. Chile Cultura y Ticketplus aportan alcance nacional; las demás agregan profundidad territorial o temática.

Fuente	Cobertura	Método principal
Chile Cultura	Nacional, 16 regiones	API pública paginada

Fuente	Cobertura	Método principal
Maipú en Común	Región Metropolitana	API de aplicación mediante Playwright
Centro Cultural GAM	Santiago	JSON-LD
Centro Cultural La Moneda	Santiago	Tarjetas HTML declarativas
Centro Cultural CEINA	Santiago	Tarjetas HTML declarativas
Valpo Cultura	Valparaíso	The Events Calendar API
Espacio Cultural Viña del Mar	Valparaíso	The Events Calendar API
Teatro Municipal de Viña del Mar	Valparaíso	JSON-LD
Agenda Cultural UOH	O'Higgins	WordPress REST
Teatro Biobío	Biobío	WordPress REST y fechas visibles
Agenda Universidad de Talca	Maule	Tarjetas HTML declarativas
Corporación Cultural de Iquique	Tarapacá	Tarjetas institucionales
Centro de Arte Molino Machmar	Los Lagos	Tarjetas institucionales
Enoturismo Chile	Interregional	WordPress/EventON
FISA	Varias regiones	Portafolio oficial
Espacio Riesco	Región Metropolitana	Tarjetas HTML declarativas
Parque Cultural de Valparaíso	Valparaíso	WordPress REST
Fundación CorpArtes	Región Metropolitana	Tarjetas HTML declarativas
Cultura Vitacura	Región Metropolitana	Tarjetas HTML declarativas
Santiago Cultura	Santiago	WordPress/EventON incremental
Ticketplus Chile	Nacional, 16 regiones	Selector regional y fichas estructuradas
FondasChile	Nacional, septiembre	Tarjetas HTML declarativas

Cada archivo en `sources/` declara URL, región, comuna, organizador, conector, categorías por defecto, límites de paginación y, cuando corresponda, metadatos estáticos de ubicación. Añadir una fuente no exige modificar el archivo principal: primero se intenta configurar un conector existente; solo se crea código nuevo cuando la estructura no coincide con ninguno.

Las fuentes retiradas se declaran en `settings.json` mediante `retired_source_names`. Durante la siguiente publicación se concilian intencionalmente sus eventos extraídos con procedencia acreditada. Se conservan publicaciones manuales, eventos suprimidos y eventos con interacciones, aunque pertenezcan a esas fuentes.

4.1. Validación de candidatas

Antes de integrar una fuente, el comando `validar_fuentes.bat` revisa el inventario de candidatas sin extraer eventos ni modificar Supabase. Comprueba disponibilidad HTTP, `robots.txt`, presencia de señales de agenda, fechas, enlaces, pertinencia cultural y cobertura chilena. Sus resultados se guardan en `data/source_validation/`; una aprobación técnica no habilita una fuente automáticamente, pues la configuración y las pruebas siguen siendo una decisión del equipo.

5. Flujo de procesamiento

5.1. Fase de extracción

1. Se cargan las fuentes habilitadas para el mes actual; FondasChile se omite fuera de septiembre.
2. Las fuentes se agrupan por dominio. Hasta cuatro dominios se procesan en paralelo, pero cada dominio conserva su propio ritmo.
3. Cada conector devuelve objetos `Event` con un esquema común.
4. Se aplican limpieza inicial, deduplicación interna y validación de disponibilidad de fichas.
5. Cada fuente exitosa o vacía de forma válida queda registrada en un checkpoint local.

La fase puede ejecutarse sin IA ni Supabase:

```
python scrape_events.py --phase extract --local-only
```

5.2. Fase de procesamiento y publicación

1. Se recupera la extracción compatible más reciente desde SQLite.
2. Se sanea el texto globalmente y se deduplican eventos entre fuentes.
3. Se aplican valores de ubicación declarados por fuente y reglas deterministas.
4. Si Supabase está habilitado, se detectan eventos nuevos, modificados, suprimidos o pertenecientes a organizadores bloqueados.
5. Solo el subconjunto pendiente pasa por OCR, redacción y enriquecimiento de ubicación.
6. Se ejecuta el filtro cultural, se descartan fechas inválidas o vencidas y se exportan JSON/CSV.
7. Se publica de forma atómica en Supabase y se reconcilian registros obsoletos de fuentes completas.

```
python scrape_events.py --phase process --supabase
```

La separación de fases permite conservar el trabajo de extracción si se agota una cuota de IA o falla temporalmente el destino remoto.

6. Modelo canónico de evento

Todo conector entrega el mismo modelo lógico. Los campos fundamentales son `title`, `start_date`, `end_date`, `venue`, `location`, `commune`, `region`, `organizer`, `categories`, `source_url` y `official_url`.

Las categorías son siempre una lista, no una cadena. Un mismo evento puede pertenecer simultáneamente a música, familiar y gratuito. En el JSON se guarda como arreglo; en CSV se serializa como JSON dentro de una celda; en Supabase se normaliza mediante `categories` y `event_categories`.

La ubicación se resuelve de forma transparente y cada evento incluye:

- `location`: texto canónico que nunca queda vacío;
- `location_precision`: dirección, recinto, comuna, región o modalidad en línea;

- `location_source`: dato publicado, configuración de fuente, regla determinista, IA o fallback geográfico.

No se inventan direcciones. Cuando la evidencia solo permite identificar una comuna o región, se conserva exactamente ese nivel y se marca la menor precisión.

7. Saneamiento, vigencia y filtro cultural

El saneador común elimina HTML residual, reglas CSS, scripts, shortcodes de WordPress/WPBakery y descripciones desproporcionadas. Esto evita que cambios visuales de un sitio se conviertan en contenido del catálogo.

Un evento se conserva solo si tiene fecha de inicio válida y todavía está vigente. La validación de URL retira fichas confirmadas con HTTP 404/410 o páginas de error inequívocas; ante timeout, bloqueo temporal o error 5xx el registro se conserva para no borrar información por una falla pasajera.

El filtro cultural funciona después de sanear y consolidar, antes de exportar. Tiene tres resultados: *aceptado*, *revisión* y *excluido*. Se incluye una actividad cuando su finalidad principal o una dimensión sustantiva permite crear, expresar, conservar, interpretar o compartir cultura. El formato por sí solo —por ejemplo "feria" o "exposición"— no determina el resultado. La auditoría queda en `data/cultural_filter_audit.json`.

8. IA, OCR y control de cuota

NVIDIA NIM se usa como respaldo, no como requisito de extracción. Las fuentes con API, JSON-LD, WordPress REST o tarjetas declarativas se procesan sin consumir IA. La IA interviene únicamente en estos casos:

- OCR de afiches cuando existe imagen y no hay texto suficiente;
- redacción editorial de texto público, preservando el texto de origen para trazabilidad;
- resolución por lotes de ubicaciones incompletas.

El límite global se configura en 40 solicitudes por minuto. Todas las llamadas, incluso las iniciadas por fuentes concurrentes, pasan por el mismo regulador. Las respuestas 429, `Retry-After` y secuencias de timeouts activan pausas controladas; los pendientes no se pierden, sino que pasan a una cola SQLite para una ejecución futura.

Antes de usar IA, Supabase compara URL original y una clave combinada de título, fecha y lugar. Si el evento ya existe y no cambió, se reutilizan OCR y texto editorial previamente almacenados. Esta deduplicación reduce costo, tiempo y consumo de cuota.

La RPC privada `match_existing_events(jsonb)` busca por clave externa, URL de extracción o URL oficial. Desde la revisión del 20 de octubre utiliza dos índices B-tree de URL normalizada y ramas de igualdad separadas. Conserva la coincidencia más reciente por clave local y resuelve empates por identificador. El cliente divide las consultas en lotes de entre 1 y 500 candidatos; el workflow utiliza 200. Esto evita respuestas masivas que puedan truncarse por el límite de filas de PostgREST, sin aumentar el límite de NVIDIA.

9. Estado local, checkpoints y recuperación

El archivo `state/pipeline.sqlite3` conserva datos operativos, no el catálogo público. Incluye checkpoints por fuente, una extracción consolidada, cola de enriquecimiento, caché de resultados de IA, métricas y caché HTTP condicional.

En GitHub Actions, el estado se guarda como artefacto después de la extracción y nuevamente al finalizar. Una ejecución siguiente restaura el último artefacto disponible. Por ello, si falla la fase de enriquecimiento, no se requiere volver a recorrer todas las páginas: la siguiente corrida retoma los eventos y pendientes conservados.

No deben ejecutarse simultáneamente dos procesos que compartan el mismo estado local o las mismas credenciales de IA. El workflow usa un grupo de concurrencia para impedir que dos ejecuciones programadas se solapen.

10. Exportación y sincronización con Supabase

El respaldo local se escribe en `data/events.json`, `data/events.csv`, archivos separados por región, `data/source_metrics.json` y `data/run_summary.json`. Al final de cada corrida se informa un total consolidado y duración completa.

La sincronización remota usa la API REST de Supabase con una clave secreta exclusiva del proceso de ingesta. El flujo remoto es:

1. transformar eventos en entidades normalizadas: fuentes, organizadores, comunas, categorías, eventos, procedencia y medios;
2. cargar fragmentos en la tabla privada `catalog_staging`;
3. ejecutar `publish_staged_catalog(...)` para publicar todo el lote dentro de una transacción;
4. conciliar ausencias e invocar `cleanup_expired_events()` dentro de esa misma transacción, si la limpieza está habilitada;
5. registrar el resultado de sincronización en `scrape_runs`.

Las consultas y cargas REST idempotentes reintentan fallas transitorias de red, timeout o disponibilidad temporal. La RPC de publicación no se repite automáticamente si su respuesta se pierde, porque podría haber finalizado y eliminado el staging; así se evita una publicación ambigua o duplicada.

Los organizadores bloqueados y los eventos suprimidos individualmente se excluyen antes de OCR, redacción y publicación. De este modo, una solicitud de retiro no se revierte en la siguiente ejecución.

10.1. Limpieza protegida y conciliación

`cleanup_expired_events()` es una función PostgreSQL privada que devuelve un `bigint`; no es un trigger ni un procedimiento invocable con `CALL`. Usa `now()` del servidor al inicio de la transacción. Evalúa tanto eventos manuales como extraídos:

- con `end_at`, elimina cuando el término es estrictamente anterior al reloj del servidor;

- sin `end_at`, elimina cuando el inicio es anterior al comienzo del día actual en `America/Santiago`; conserva todo el día local de inicio;
- conserva eventos suprimidos o con cualquier interacción de usuario, sea positiva o negativa;
- conserva fechas sin inicio o con término anterior al inicio, para revisión;
- elimina por cascada las relaciones de categorías, medios y procedencia del evento retirado.

La conciliación por ausencia es otra regla: requiere una fuente autorizada, una fila no vista en la sincronización actual, procedencia en `event_provenance` y un método distinto de `manual`. También protege supresiones e interacciones. Puede retirar una ficha con fecha futura si ya no aparece en una fuente completa; una fuente fallida o parcial no autoriza esa conciliación.

Las dos rutas utilizan un bloqueo `SHARE` de `user_event_interactions` para evitar perder señales concurrentes; el tiempo de espera de bloqueos se limita a cinco segundos. Una falla revierte publicación, relaciones y borrados, manteniendo staging. Las inserciones de interacciones pueden esperar durante esta transacción.

La publicación devuelve `removed_stale` y `removed_expired`. El exportador guarda esos contadores en `scrape_runs.metadata` cuando marca el éxito. No son el total restante del catálogo. La escritura final de la bitácora ocurre después de la RPC y no revierte una publicación ya confirmada si falla.

La conservación de eventos con interacciones es temporal, hasta desacoplar las señales del recomendador. El frontend debe seguir filtrando las fechas: una fila conservada por protección no necesariamente representa un evento vigente. Las RPC de limpieza y publicación solo admiten ejecución con `service_role`; `anon` y `authenticated` no tienen ese permiso.

11. Configuración

La configuración sensible vive en `.env` y nunca debe subirse al repositorio. Las variables mínimas para sincronizar son:

```
SUPABASE_ENABLED=true
SUPABASE_URL=https://TU_PROYECTO.supabase.co
SUPABASE_SECRET_KEY=sb_secret_...
NVIDIA_API_KEY=...
```

Los parámetros operativos más relevantes son:

Variable	Finalidad
`SOURCE_WORKERS`	Dominios procesados en paralelo; valor recomendado: 4.
`HTTP_CACHE_ENABLED`	Activa revalidación condicional mediante ETag y Last-Modified.
`NVIDIA_RPM_LIMIT`	Límite global de solicitudes a NVIDIA; configurado en 40.
`NVIDIA_OCR_MAX_IMAGES`	Máximo de afiches OCR por ejecución.
`NVIDIA_REWRITE_ENABLED`	Activa o desactiva la redacción editorial.

Variable	Finalidad
SUPABASE_DELETE_EXPIRED	Elimina eventos ya terminados al sincronizar.
SUPABASE_RECONCILE_STALE	Retira eventos no vistos por fuentes completas.
SUPABASE_BATCH_SIZE	Fragmentos REST; 200 en el workflow. La consulta de coincidencias limita sus lotes a 500 como máximo.
SUPABASE_MAX_RETRIES	Reintentos de operaciones REST idempotentes ante fallas transitorias.
SUPABASE_RETRY_MAX_WAIT_SECONDS	Espera máxima entre reintentos REST.

12. Automatización con GitHub Actions

El workflow separa validación y ejecución en dos trabajos sobre `ubuntu-24.04`. El trabajo `validate` instala dependencias, ejecuta las pruebas de Python y utiliza PostgreSQL 17 temporal para aplicar todas las migraciones y probar las regresiones SQL. El contenedor existe únicamente en GitHub Actions: no requiere instalar Docker en el computador local ni contenerizar el scraper.

El trabajo `scrape` depende del éxito de `validate`. Instala Chromium, restaura checkpoints, extrae, conserva un artefacto, enriquece y sincroniza con limpieza protegida. Conserva las copias de estado al finalizar aunque una fase falle. Las credenciales se almacenan como *Repository secrets*: `SUPABASE_URL`, `SUPABASE_SECRET_KEY` y `NVIDIA_API_KEY`. La validación SQL no utiliza estas claves.

La programación vigente es diaria a las 23:37 en la zona `America/Santiago`, mediante `37 23 * * *`. El workflow conserva `workflow_dispatch` para ejecución manual. La zona horaria evita ajustar el cron por horario de verano; el programador de GitHub puede retrasar el inicio. Las migraciones remotas se aplican por separado y no en cada ejecución programada.

12.1. Pruebas de base de datos

`tools/test_database.sh` exige una base vacía llamada `eventos_test`, instala un contrato mínimo ficticio de Auth y aplica las migraciones ordenadas antes de ejecutar `supabase/tests/*.sql`. No lee `.env` y nunca debe apuntar al proyecto de producción. El fixture comprueba el comportamiento del catálogo, no el servicio real de autenticación.

Las regresiones cubren vencimiento, día chileno, fechas incoherentes, supresiones, interacciones, cascadas, permisos, idempotencia, coincidencias por clave/URL y rollback de publicación ante un fallo forzado. El 20 de octubre pasaron las 144 pruebas de Python, incluida la comprobación de cobertura documental de migraciones, y las tres suites SQL locales sobre PostgreSQL embebido con PGlite, en UTC, Santiago y Tokio. La ejecución del nuevo trabajo con PostgreSQL 17 en Actions se verifica después de subir el cambio; no se presenta como una prueba remota ya realizada.

12.2. Cambios del 20 de octubre de 2026

- `20261020000100`: crea `cleanup_expired_events()` y restringe su ejecución al proceso de ingesta.
- `20261020000200`: incorpora índices para URL de origen y oficial. Optimiza `match_existing_events(jsonb)` sin modificar su firma.
- `20261020000300`: conecta la limpieza protegida con la publicación atómica y protege las publicaciones manuales frente a la conciliación.

Las nueve migraciones locales y remotas quedaron sincronizadas. Su instalación no ejecutó el scraper ni eliminó eventos; el nuevo comportamiento opera en la siguiente publicación correcta. No se añadieron tablas, columnas ni relaciones, por lo que el modelo ER permanece vigente.

13. Operación y mantenimiento

13.1. Comandos frecuentes

```
# Instalacion en Windows
instalar.bat

# Ejecucion completa local
ejecutar.bat

# Pruebas
python -m unittest discover -s tests -v

# Solo extraccion, sin IA ni Supabase
python scrape_events.py --phase extract --local-only

# Procesar un checkpoint y sincronizar
python scrape_events.py --phase process --supabase

# Aplicar migraciones de Supabase en Windows
migrar_supabase.bat
```

13.2. Cuando una fuente cambia o cae

1. revisar el resumen de fuente y confirmar si falló, quedó parcial o está legítimamente vacía;
2. comprobar la página pública y sus términos de uso antes de cambiar la extracción;
3. preferir API, JSON-LD o endpoint público sobre selectores visuales frágiles;
4. actualizar solo el JSON de fuente cuando un conector declarativo sea suficiente;
5. agregar o corregir pruebas con una muestra representativa antes de habilitar la fuente;
6. si la fuente falla, mantenerla fuera de la reconciliación: el pipeline no borra su catálogo por una extracción parcial o fallida.

La guía operativa detallada está en `sources/GUIA_AGREGAR_Y_MANTENER_FUENTES.md`.

14. Seguridad, límites y consideraciones

Se consultan únicamente recursos públicos. Antes de añadir una fuente se deben revisar términos de uso, `robots.txt`, condiciones de reutilización y frecuencia razonable de consulta. El sistema no debe intentar evadir bloqueos ni acceder a contenido privado.

Las claves secretas de Supabase se usan solamente por el scraper o GitHub Actions; nunca deben llegar al frontend, a archivos versionados ni a logs. El OCR y la redacción automatizada no sustituyen una revisión editorial ni garantizan, por sí solos, conformidad con políticas de terceros.

15. Documentos relacionados

- `README.md`: instalación, comandos cotidianos y visión rápida.
- `docs/SOURCES.md`: procedencia y tratamiento de cada fuente.
- `docs/DATABASE.md`: diccionario de la base de datos, mantenido por separado.
- `sources/GUIA_AGREGAR_Y_MANTENER_FUENTES.md`: operación detallada de fuentes.
- `docs/ESTANDAR_DOCUMENTACION.md`: formato común para documentos y diagramas del proyecto.
- `docs/historico/`: auditorías y validaciones fechadas como evidencia.

Fin de la documentación técnica del scraper.